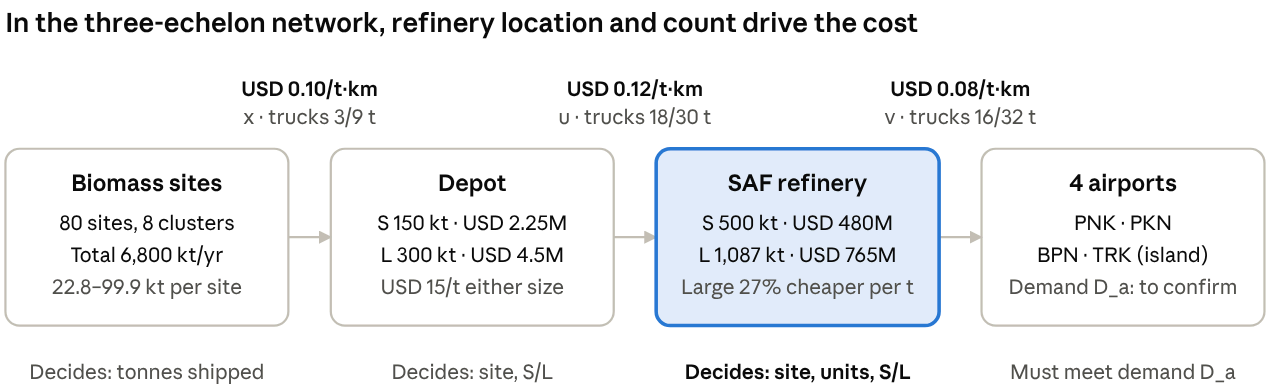
Kalimantan SAF Supply Chain Optimisation: Modelling Plan and Team Roles

Oct 31, 2026 · @Alexi

1. Goal and scope

We will build a **multi-echelon capacitated facility location model (multi-echelon CFLP, solved as a MILP)**. It moves 6,800 kt/year of biomass from 80 sites through pre-processing depots and SAF refineries to Kalimantan's 4 international airports, and minimises **annualised CapEx + OpEx + transport cost**.

This document is the design plan before any build. For each stage it sets out what we do, which options we compare, the recommended decision, and the data feature or formula behind it. The statistics in section 2 were computed on the two raw data files in the project folder; the model has not been solved yet.



Supply chain network · figures from the brief's facility table, transport table and biomass CSV

Covering 6,800 kt on an input basis, refineries are about 98% of CapEx (about 92% on an output basis at 20% yield) and depots the rest. So depot locations are driven mainly by transport, while refinery location and count come from the trade-off between CapEx and transport.

The 5 questions the model answers

Decision	How the model expresses it	Delivered as
Where to build depots, and how many	0-1 variables on candidate sites, then coordinate refinement after solving	Depot coordinate table (lat/lon)
Small or Large depot	Size s ∈ {Small 150 kt, Large 300 kt}	Size and utilisation table
Where to build refineries, how many, how large	Integer variables on candidate sites (several units per site allowed)	Refinery coordinate and size table
Tonnes moved on each leg	Continuous flows x (farm → depot), u (depot → refinery), v (refinery → airport)	Flow table, map flow lines, Sankey diagram
Small or large trucks on each leg	Trips and fleet size derived from flows after solving	Fleet and trip table

Scope

- **In scope**: 4 airports (PNK Pontianak, PKN Pangkalan Bun, BPN Balikpapan, TRK Tarakan), 80 biomass sites, 2 facility types × 2 sizes, 3 road-truck legs.
- **Out of scope (listed as extensions)**: river or sea transport, phased build-out, carbon constraints, demand growth over time.

Final deliverables

1. A reproducible Python pipeline: data → distance matrices → candidate sites → MILP → results
2. The optimal plan: facility coordinates and sizes, flows on each leg, cost breakdown
3. An interactive map and a material-flow Sankey diagram
4. Sensitivity analysis: annualisation, conversion yield, road circuity factor, demand split, capacity basis
5. Report and presentation deck

2. Datasets and key features

Both files are clean (no missing values, no duplicates), but the airport names hide 3 matching traps. The biomass sits in 8 tight district clusters, with 70.5% in the west. These two facts drive the siting method in the later stages.

2.1 airports.dat (OpenFlights format)

- 7,697 rows, no header, 14 columns: id, name, city, country, IATA, ICAO, lat, lon, altitude, timezone, DST, tz, type, source. Missing values are written as `\N`.
- The model uses only 3 features: IATA, lat, lon.
- Name-matching traps:
  - Searching "Iskandar" also returns Sultan Iskandar Muda International Airport (Banda Aceh, Sumatra, BTJ), which is wrong.
  - The brief says "Sultan Aji Muhammad Sulaiman Sepinggan"; the file says "Sultan Aji Muhamad Sulaiman Airport" (one m fewer, no Sepinggan), so a full-name search finds nothing.
  - Juwata's city field is spelled "Taraken" (should be Tarakan).
- Decision: **filter by IATA code**, then check the coordinates fall inside a Kalimantan bounding box.

IATA	Name in file	City	Lat	Lon
PNK	Supadio Airport	Pontianak	-0.1507	109.4040
PKN	Iskandar Airport	Pangkalan Bun	-2.7052	111.6730
BPN	Sultan Aji Muhamad Sulaiman Airport	Balikpapan	-1.2683	116.8940
TRK	Juwata Airport	Tarakan (island)	3.3267	117.5694

Straight-line distances between neighbouring airports: PNK–PKN 380 km, PKN–BPN 602 km, BPN–TRK 516 km; the farthest pair, PNK–TRK, is 986 km. TRK is on Tarakan Island, with no road link to the mainland.

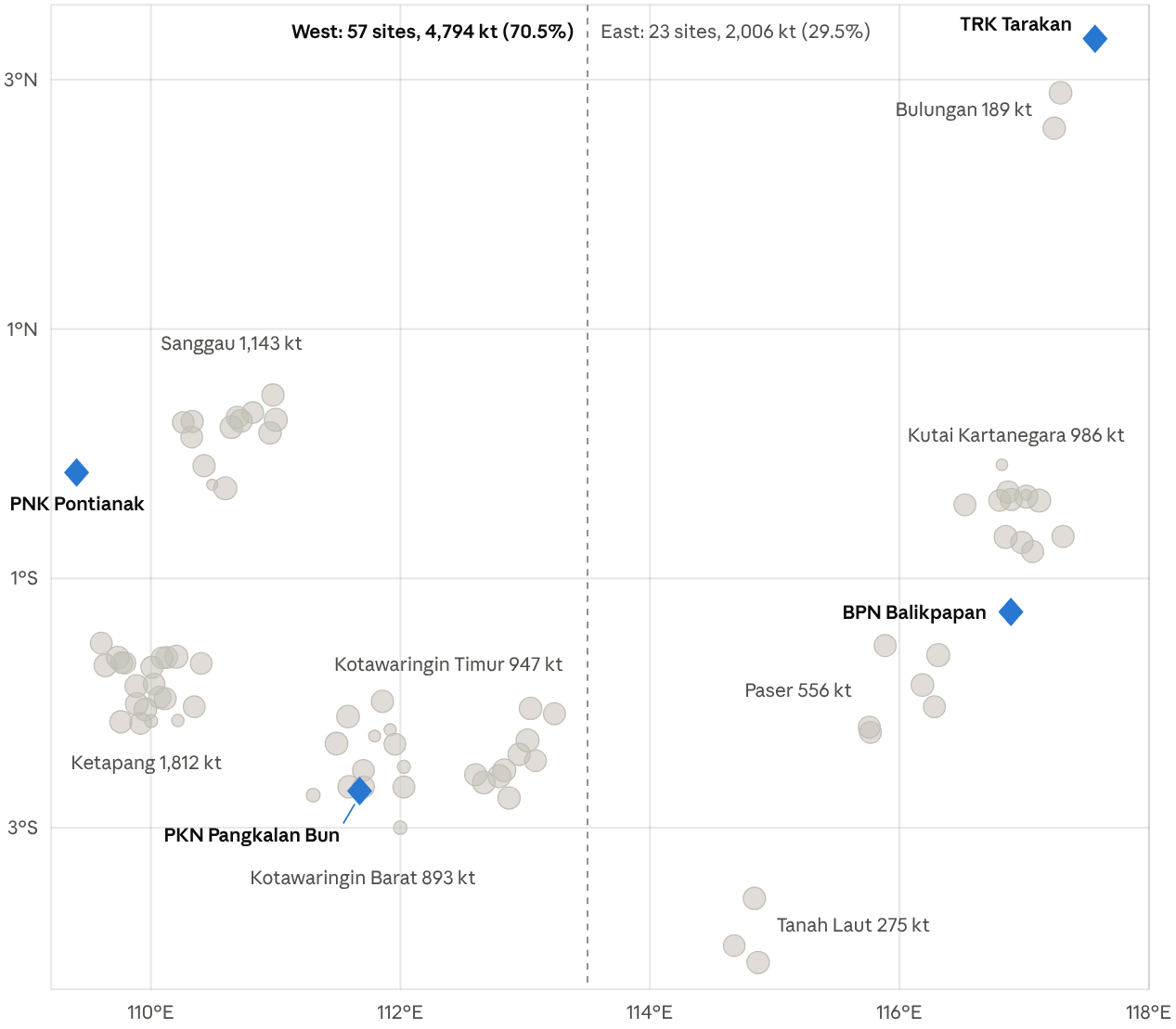
2.2 Biomass CSV

- 80 rows × 7 columns: location_id, province, district, production_kilotons, production_tonnes, latitude, longitude.

- Quality checks: no missing values; no duplicate IDs or coordinates; production_tonnes = kilotons × 1000 on every row; the total is exactly 6,800 kt.
- **Bimodal distribution**: 70 sites at 80–100 kt and 10 sites at 22.8–35.9 kt, with none between 40 and 80 kt. Mean 85 kt, standard deviation 21.7 kt.
- Covers 5 provinces and **8 districts**.

70.5% of biomass lies west of 113.5°E, in 8 clusters hundreds of km apart

Grey: 80 sites, bubble area by output (22.8–99.9 kt) · Blue: 4 airports · equal lat/lon scale



kalimantan_80_locations_6800kt_full_range.csv + airports.dat (filtered by IATA) · 80 biomass sites, 4 airports

The four western districts (Ketapang, Sanggau, both Kotawaringins) form one band, with PKN inside the Kotawaringin Barat cluster. In the east, Kutai Kartanegara and Paser sit either side of BPN. Near TRK there are only Bulungan's 2 sites.

District	Province	Sites	Biomass (kt)	Share	Weighted centroid (lat, lon)	Farthest site from centroid (km)
Ketapang	West	21	1,811.8	26.6%	-1.829, 109.967	53
Sanggau	West	13	1,142.9	16.8%	0.178, 110.647	51
Kutai Kartanegara	East	12	985.7	14.5%	-0.486, 116.946	47
Kotawaringin Timur	Central	10	946.6	13.9%	-2.441, 112.908	54
Kotawaringin Barat	Central	13	893.2	13.1%	-2.438, 111.744	69
Paser	East	6	556.2	8.2%	-1.905, 116.038	47
Tanah Laut	South	3	274.7	4.0%	-3.863, 114.797	33
Bulungan	North	2	189.0	2.8%	2.754, 117.268	16

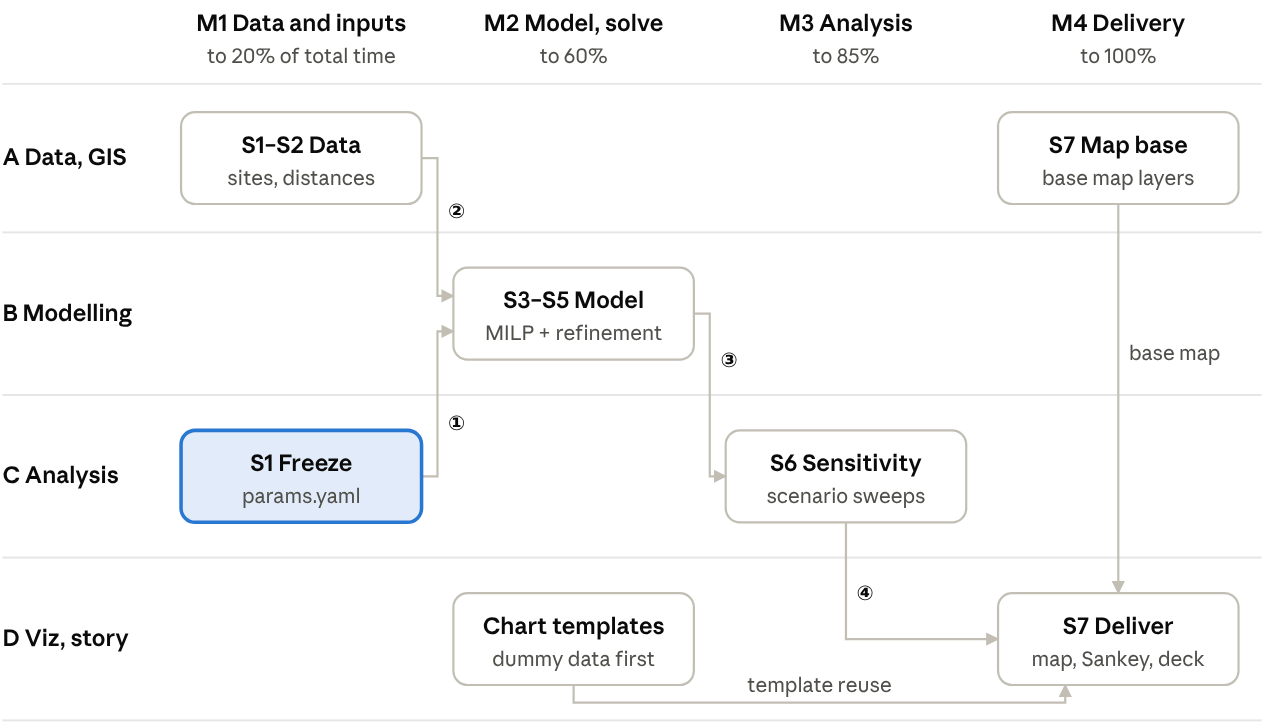
2.3 Relationships in the data (the evidence for later decisions)

1. **Close within clusters, far between them**: the median nearest-neighbour distance between sites is 13.1 km (max 48 km), while the median distance between any two sites is 342 km (max 1,008 km). No site is more than 69 km from its district centroid. This supports "depots inside clusters, refineries at cluster level".
2. **East–west imbalance**: west of 113.5°E there are 57 sites with 4,794 kt (70.5%), weighted centroid (-1.585, 111.041); east of it, 23 sites with 2,006 kt (29.5%), weighted centroid (-1.037, 116.430). The overall weighted centroid (-1.423, 112.630) falls in the empty gap between the two, so "one central refinery" is a wrong intuition.
3. **Airports and feedstock don't line up**: assigning each site to its nearest airport by straight line gives PKN 34 sites / 2,724 kt, PNK 24 / 2,158 kt, BPN 20 / 1,729 kt, and TRK only 2 / 189 kt. TRK's SAF will mostly have to come from far away.
4. **Site size vs facility capacity**: every site is ⩽ 100 kt, below a Small depot's 150 kt; one Large depot (300 kt) takes about 3 large sites. Processing everything needs at least the capacity of 23 Large or 46 Small depots.
5. **Inside the cost table**: both depot sizes cost 15 USD per tonne of capacity (no economies of scale); refineries cost 960 USD/t Small vs 704 USD/t Large (Large is 27% cheaper). Transport rates are the same for both truck sizes.

3. Team roles

The plan assumes 4 people, each owning one workstream, joined by 4 fixed handoffs. With 3 people, merge C and D; with 5, split B into "candidate sites and coordinate refinement" and "MILP and solving".

4 parallel workstreams; 4 handoffs link the 7 stages



① params.yaml frozen · ② site tables and distance matrices
③ first feasible solution · ④ solution.json and scenarios.csv
Blue box = critical gate: no model work until parameters are frozen

Team swimlane · 4 roles, 4 milestones, 4 handoffs

B cannot start modelling until both ① and ② arrive. D does not wait for results: during M2 it builds the chart templates on dummy data.

Role	Stages owned	Core tasks	Deliverables	Tools / skills
A Data & GIS	1, 2, and the map base layer for 7	Clean data, filter airports by IATA, compute distance matrices, calibrate the circuity factor with OSRM samples	farms_clean.csv, airports4.csv, three distance matrices	pandas, NumPy, geopy, OSRM
B Optimisation modelling	3, 4, 5	Generate candidate sites, write the MILP, tune the solver, run the coordinate refinement loop	candidates.csv, model.py, solution.json	Pyomo/PuLP, HiGHS/Gurobi, scikit-learn
C Parameters & analysis	Assumption freeze in 1, and 6	Research conversion yield, OpEx, airport demand; keep the assumption register; run sensitivity and baseline comparisons	params.yaml, assumptions.md, scenarios.csv	Literature search, experiment design, pandas
D Visualisation & storytelling	7, report	Interactive map, Sankey, cost charts, presentation storyline and deck	map.html, charts, deck, report	folium, plotly, presentation tools

Handoffs

1. **C → whole team (end of Stage 1)**: freeze params.yaml. After this, parameter changes go through C, and the model reads parameters only from this file.
2. **A → B (end of Stage 2)**: cleaned site tables + distance matrices.
3. **B → C (end of Stage 5)**: the first feasible solution; C starts running scenarios with the same model function.
4. **B, C → D (end of Stage 6)**: solution.json + scenarios.csv; D produces the charts. D can build the chart templates earlier with dummy data.

Suggested repository layout

```
saf-kalimantan/
  data/raw/            original airports.dat and biomass CSV (read-only)
  data/clean/          farms_clean.csv, airports4.csv, dist_*.parquet
  config/params.yaml   all costs, capacities, yields, demand, annualisation
inputs
  src/01_clean.py
  src/02_distance.py
  src/03_candidates.py
  src/04_model.py      build_model(params, candidates) -> model object
  src/05_solve.py
  src/06_scenarios.py
  src/07_viz.py
  outputs/             solution.json, scenarios.csv, map.html, charts
```

4. Stage 1 Data preparation and assumption freeze

This stage produces clean input tables and fills the parameters the brief leaves out before modelling starts. The **capacity basis** comes first: it changes the number of refineries by a factor of 3–5.

Owners: A (data), C (parameters)

Tasks

1. Read airports.dat (header=None, name the 14 columns, na_values='\N'), filter IATA ∈ {PNK, PKN, BPN, TRK}, and check the coordinates fall within lat ∈ [-4.5, 4.5], lon ∈ [108, 119.5].
2. Read the biomass CSV and assert: total = 6,800 kt, tonnes = kt × 1000, coordinates in range. Standardise units to t/year.

3. Write the missing parameters into params.yaml and log each in assumptions.md: value, source, sensitivity range.

Output (what this delivers): farms_clean.csv, airports4.csv, params.yaml, assumptions.md. Every later script reads only from these files.

Options compared and decisions

Decision	Options	Recommendation	Evidence
Airport filter	① fuzzy name match ② IATA code ③ type coordinates by hand	②	Names have 3 traps (section 2.1); IATA is unique and reproducible
Capacity basis	① table capacity = biomass input ② = SAF output	Ask the organisers first; run both through a capacity_basis switch	Under ①, 6,800 kt needs at least 7 refineries (6 Large + 1 Small = 7,022 kt, CapEx USD 5.07 bn); under ② with 20% yield, SAF is 1,360 kt and needs only 1 Large + 1 Small (USD 1.245 bn)
CapEx treatment	① add one-off CapEx to annual costs ② annualise with the capital recovery factor (CRF) ③ full-life NPV	② as the main case, ③ in the report for comparison	① adds USD to USD/year, which is dimensionally inconsistent and lets CapEx swamp transport
Use all 6,800 kt?	① must use all (equality) ② use as demand requires (≤)	Write ≤ with a switch	When demand is below capacity, the model drops the most remote sites automatically
Airport demand	① given by organisers ② split by each airport's passenger traffic or fuel uplift ③ equal split	① > ② > ③	③ has no data behind it and is only a comparison scenario; C researches the data for ②

Decision	Options	Recommendation	Evidence
OpEx	① annual % of CapEx ② USD per tonne processed ③ ignore	① + ② combined	Fixed OpEx joins the facility fixed cost; variable OpEx multiplies flow. Values still to be researched

One clue on capacity basis: 6,800 × 0.16 = 1,088, almost exactly the Large refinery's 1,087 kt. The setters may have designed it as "output basis + 16% yield", but this is a guess to confirm with the organisers.

CapEx annualisation (r = discount rate, n = facility life; the example uses r = 8%, n = 20):

```
\mathrm{CRF}(r,n)=\frac{r(1+r)^n}{(1+r)^n-1},\qquad \mathrm{CRF}
(0.08,20)\approx 0.1019
```

```
\text{Annualised CapEx}_{\text{Large refinery}} = 765\,\text{M} \times 0.1019
\approx 77.9\,\text{M USD/year}
```

Tools: pandas (read_csv, assertions), PyYAML (parameter file), pytest (data checks as tests, so any data change fails loudly).

5. Stage 2 Distance and transport cost matrices

The main approach is **Haversine straight-line distance × a circuity factor τ**, with τ calibrated on a sample of OSRM road routes. The key reason: Stage 3 refines coordinates in continuous space, and that iteration needs a closed-form distance function.

Owner: A

Tasks

1. Compute three distance matrices (km): sites × depot candidates (80 × J), depot candidates × refinery candidates (J × K), refinery candidates × airports (K × 4).
2. Route 30–50 point pairs with OSRM and fit τ = road distance / straight-line distance; check short and long trips separately.
3. Multiply by each leg's rate to get per-tonne transport cost matrices (USD/t).
4. Treat TRK separately: it is on an island, so add a ferry or barge cost.

Output (what this delivers): three distance matrices dist_fd / dist_dr / dist_ra, the matching cost matrices, and a reusable dist(lat1, lon1, lat2, lon2) function.

Formulas

Haversine (R = 6,371 km, φ = latitude, λ = longitude, in radians):

```
d_{ij}=2R\arcsin\sqrt{\sin^2\frac{\varphi_j-\varphi_i}
{2}+\cos\varphi_i\cos\varphi_j\sin^2\frac{\lambda_j-\lambda_i}{2}}
```

Per-tonne transport cost (the three rates come from the brief's transport table):

```
c^{FD}_{ij}=0.10\,\tau\,d_{ij},\qquad c^{DR}_{jk}=0.12\,\tau\,d_{jk},\qquad
c^{RA}_{ka}=0.08\,\tau\,d_{ka}\quad(\text{USD/t})
```

Options compared

Option	Accuracy	Effort	Verdict
① Euclidean distance on raw lat/lon	Poor; units are degrees, not km	Lowest	Not used
② Haversine straight line	Systematically underestimates road distance	Low	Base
③ Haversine × τ	Medium, calibratable	Low; τ can go into sensitivity	**Main approach**
④ OSRM (OpenStreetMap road network)	High, but rural roads may be missing	Medium; needs hosting or an API	Calibrate τ on samples; re-check the final plan
⑤ Google Distance Matrix	High	Paid, with quotas	Not used

Evidence from the data

- The median nearest-neighbour distance between sites is 13.1 km, so farm → depot legs are short; airports are 380–986 km apart, so refinery → airport legs may be long. Errors in τ mostly hit the long legs, so τ goes into the sensitivity analysis.
- Small and large trucks have the same rate, so the cost matrices do not depend on truck type; truck choice is derived from flows in Stage 6.

- Scale: moving 1 t over 100 km costs USD 10, 12 and 8 on the three legs.

Handling the TRK sea crossing: ① ignore the water (underestimates) ② add a fixed ferry/barge cost parameter ③ assume shipping via BPN. Recommend ②, with the parameter in params.yaml and stated in the report.

Tools: vectorised Haversine in NumPy (whole matrix in one call), the OSRM HTTP API or a local Docker instance, SciPy linear regression (to fit τ).

6. Stage 3 Candidate site generation and coordinate refinement

Depot candidates = the 80 site locations + weighted k-means centroids per district. Refinery candidates = the 8 district centroids + the Weber points of the east and west macro-clusters + the 4 airports. After the MILP solves, the Weiszfeld algorithm refines coordinates and the MILP is re-solved until cost stops falling. This produces the "optimal coordinates" the brief asks for.

Owner: B (A supplies the distance function)

Tasks

1. Generate the depot candidate set J (about 140) and refinery candidate set K (about 15).
2. Write a Weiszfeld function: given a facility's inbound and outbound flows, find the coordinate that minimises weighted transport cost.
3. Write the alternating location-allocation loop (below) and integrate it with Stages 4 and 5.

Output (what this delivers): candidates.csv (type, coordinates, generating method), weiszfeld(), refine_loop().

Depot candidates: options compared

Option	Pros	Cons	Verdict
① The 80 site locations only	Simple, real coordinates	The optimum often lies between sites	Used as a subset
② Regular grid (0.1° ≈ 11 km)	Full coverage	The island spans about 7° × 8°, roughly 5,600 points, most with no feedstock; variables explode	Not used

Option	Pros	Cons	Verdict
③ Weighted k-means centroids per district	Follows the feedstock	k-means minimises squared distance, not transport cost	Used as a subset
④ ① + ③ combined	Dense candidates, small model (J ≈ 140)	—	**Main approach**

Evidence: a 13.1 km median nearest-neighbour distance and a district radius of ⩽ 69 km mean candidates are dense enough. Depots are only about 2–8% of CapEx, so extra candidates add no cost risk. Every site is ⩽ 100 kt < 150 kt, so each district's k-means K ranges from ⌈district total / 300⌉ to ⌈district total / 150⌉.

Refinery candidates: options compared

Option	Scenario it represents	Verdict
① The 8 district weighted centroids	Local processing, distributed plants	Used
② Weber points of the west cluster (-1.585, 111.041) and east cluster (-1.037, 116.430)	Centralised plants that exploit economies of scale	Used
③ The 4 airport locations	Close to demand	Used, to test the "close to feedstock" conclusion
④ Ports, industrial estates	Real-world buildability	Needs extra data; listed as an extension
⑤ All 80 site locations	Robustness check	Optional; still solves fast

Several refinery units may be built at one candidate site (integer variable). Evidence: Ketapang alone has 1,812 kt, more than one Large unit (1,087 kt) on an input basis.

Why refineries are pulled towards feedstock (Weber's weight-losing principle)

Let y be the conversion yield (t SAF / t depot output) and α the depot mass ratio (t depot output / t biomass). Per tonne of biomass per km, shipping the product (SAF) costs 0.08αy and shipping the intermediate (depot output) costs 0.12α. Whenever the following holds, the refinery should sit on the feedstock side:

$$0.08\,\alpha y < 0.12\,\alpha \iff y < 1.5$$

A yield cannot exceed 1, so this holds for any yield. In the same way, a depot trades off the farm side (0.10 per t·km) against the refinery side (0.12α per t·km):

$$\text{depot is pulled towards farms} \iff 0.12\,\alpha < 0.10 \iff \alpha < 0.833$$

In other words, if pre-processing (drying, pelletising) cuts mass by more than 16.7%, depots should sit next to the farms; otherwise they shift slightly towards the refinery. α is a parameter still to be confirmed.

Continuous coordinates: options and algorithm

Option	Character	Verdict
① Choose among candidates only (discrete)	Simple; coordinates limited to candidates	First pass
② Alternating location-allocation (Cooper) + Weiszfeld	Local optimum, easy to implement	**Main approach**
③ MINLP with continuous coordinates + integers	Optimal in theory, but non-convex and hard to solve	Not used

Single-facility Weber problem (w_i = flow × leg rate, p_i = the points connected to the facility):

$$\min_{p}\;\sum_i w_i\, d(p, p_i)$$

Weiszfeld iteration (computed on a local equirectangular projection):

$$p^{(t+1)}=\frac{\sum_i w_i\,p_i / d\big(p^{(t)},p_i\big)}{\sum_i w_i / d\big(p^{(t)},p_i\big)}$$

The alternating loop:

1. Solve the MILP on the current candidate set to get open facilities and flows.
2. For each open facility, run Weiszfeld on its inbound and outbound flows to get a new coordinate.
3. Add the new coordinates to the candidate set and re-solve the MILP.

4. Stop when total cost improves by less than 0.1%. Because old candidates are always kept, cost can only fall from one round to the next.

Tools: scikit-learn KMeans (sample_weight = production), Weiszfeld in NumPy, pyproj or an equirectangular projection for local planar coordinates.

7. Stage 4 MILP formulation

The model is a MILP of "three-echelon network flow + facility opening". Every cost is linear in flow, and economies of scale are captured by the discrete Small/Large sizes, so no non-linear modelling is needed. At about 2,000–11,000 continuous variables and about 300 integer variables, an open-source solver can handle it.

Owner: B; **Output**: build_model(params, candidates) in model.py, with every parameter read from params.yaml.

Sets

- I: biomass sites (80)
- J: depot candidates (about 140)
- K: refinery candidates (about 15)
- A: airports {PNK, PKN, BPN, TRK}
- S: sizes {Small, Large}

Parameters and data sources

Symbol	Meaning	Value	Source
s_i	Annual supply at site i (t/year)	22,840–99,900	CSV production_tonnes
Q^D_s	Depot capacity (t/year)	150,000 / 300,000	Facility table
Q^R_s	Refinery capacity (t/year)	500,000 / 1,087,000	Facility table
F^D_s	Depot CapEx (USD)	2.25M / 4.5M	Facility table
F^R_s	Refinery CapEx (USD)	480M / 765M	Facility table
CRF	Capital recovery factor	0.1019 (r = 8%, n = 20)	Assumption

Symbol	Meaning	Value	Source
φ^D, φ^R	Fixed OpEx (annual share of CapEx)	To research	Assumption
o^D, o^R	Variable OpEx (USD/t)	To research	Assumption
d_ij, d_jk, d_ka	Road distance (km)	Haversine × τ	airports.dat + CSV coordinates
c1, c2, c3	Leg rates (USD/t/km)	0.10 / 0.12 / 0.08	Transport table
α	Depot mass ratio	To research (default 1)	Assumption
y	SAF conversion yield	To research	Assumption
D_a	Annual SAF demand at airport a (t)	To research	Assumption
M_k	Max refinery units per candidate site	3	Assumption

Decision variables

- x_ij ⩾ 0: annual tonnes from site i → depot j
- u_jk ⩾ 0: annual tonnes from depot j → refinery k
- v_ka ⩾ 0: annual tonnes from refinery k → airport a
- z_js ∈ {0, 1}: build a depot of size s at j
- w_ks ∈ {0, 1, 2, …}: number of refinery units of size s at k

Objective (annualised total cost, USD/year)

```
\min Z=\underbrace{\sum_{j,s}(\mathrm{CRF}+\phi^D)F^D_s z_{js}+\sum_{k,s}
(\mathrm{CRF}+\phi^R)F^R_s w_{ks}}_{\text{CapEx + fixed
OpEx}}+\underbrace{o^D\sum_{i,j}x_{ij}+o^R\sum_{j,k}u_{jk}}_{\text{variable
OpEx}}+\underbrace{\sum_{i,j}c_1 d_{ij}x_{ij}+\sum_{j,k}c_2
d_{jk}u_{jk}+\sum_{k,a}c_3 d_{ka}v_{ka}}_{\text{transport}}
```

Constraints

(1) Supply: each site ships no more than it produces. If all 6,800 kt must be used, change ⩽ to =.

$$\sum_{j\in J}x_{ij}\le s_i\quad\forall i\in I$$

(2) Depot mass balance: inflow × α = outflow.

$$\alpha\sum_{i\in I}x_{ij}=\sum_{k\in K}u_{jk}\quad\forall j\in J$$

(3) Depot capacity, and (4) at most one size per candidate site:

$$\sum_{i\in I}x_{ij}\le\sum_{s\in S}Q^D_s z_{js},\qquad \sum_{s\in S}z_{js}\le 1\quad\forall j\in J$$

(5) Refinery mass balance: inflow × y = SAF output.

$$y\sum_{j\in J}u_{jk}=\sum_{a\in A}v_{ka}\quad\forall k\in K$$

(6) Refinery capacity, chosen by the Stage 1 capacity-basis switch (left: input basis; right: output basis), and (7) the unit cap per site:

$$\sum_{j}u_{jk}\le\sum_{s}Q^R_s w_{ks}\;\;\text{or}\;\;
y\sum_{j}u_{jk}\le\sum_{s}Q^R_s w_{ks},\qquad \sum_{s}w_{ks}\le M_k\quad\forall k\in K$$

(8) Airport demand:

$$\sum_{k\in K}v_{ka}\ge D_a\quad\forall a\in A$$

(9) Strengthening constraint (does not change the optimum; tightens the LP relaxation and speeds up solving): no deliveries to a site with no depot.

$$x_{ij}\le s_i\sum_{s\in S}z_{js}\quad\forall i\in I,\,j\in J$$

Optional extensions:

- Single sourcing: each site supplies only one depot (closer to real contracts); adds 80 × J binary variables.
- Maximum haul distance: no x_ij variable where d_ij exceeds a threshold (e.g. 150 km). With cluster radii ⩽ 69 km this removes no sensible option, and it cuts x from 11,200 to about 2,000 variables.

- Per-trip fixed cost: make truck type a variable only if the organisers give a per-trip charge.

Why a MILP

Option	Pros	Cons	Verdict
① Heuristics (nearest assignment, greedy opening)	Fast, intuitive	No optimality guarantee	Used as the baseline
② Exact MILP	Proof of optimality (gap), interpretable constraints	Costs must be linear	**Main approach**
③ Metaheuristics (genetic algorithm, simulated annealing)	Handle non-linearity	Lots of tuning, no lower bound	Not used
④ Two-stage decomposition (refineries first, then depots)	Simple	Ignores coupling between echelons; sub-optimal	Only to build a starting solution

Tools: Pyomo (modelling language, solver-agnostic) or PuLP (lighter).

8. Stage 5 Solving and validation

Solve with HiGHS at a 0.5% MIP gap, then validate with 6 checks and a "nearest-greedy" baseline. This stage delivers more than a plan: it states how far the plan can be from the theoretical optimum, and how much it saves against the intuitive plan.

Owners: B solves, C reviews

Tasks

1. **Numerical scaling**: flows in kt, costs in USD million. Otherwise coefficients span 10 orders of magnitude (765,000,000 down to 0.08) and the solver may hit numerical trouble.
2. Solve the LP relaxation first for a lower bound, then the MILP. Set mip_rel_gap = 0.005, time_limit, threads.
3. Warm-start from the baseline solution to cut solve time.

4. Run the Stage 3 alternating refinement loop.
5. Run the checks and export solution.json.

Output (what this delivers): solution.json (facility coordinates, sizes, flows on each leg, cost breakdown, gap), a validation report, and the comparison with the baseline.

Solvers compared

Solver	Licence	Character	Verdict
HiGHS	Open source (MIT)	Usually the fastest open-source MILP solver; supported by Pyomo and PuLP	**Main solver**
CBC	Open source (EPL)	Bundled with PuLP, slower	Backup
Gurobi	Commercial, free for academics	Fastest	Use for large scenario batches if an academic licence is available
OR-Tools CP-SAT	Open source	Integers only; continuous flows must be scaled and rounded	Not used

Baseline (nearest-greedy)

1. Build depots at each district's k-means centroids, sized by production.
2. Open refineries at district centroids in descending order of district output until capacity is sufficient.
3. Send every flow to the nearest next-echelon facility; each airport is supplied by its nearest refinery.

The report states how much the MILP saves against the baseline (in %), and in which cost line (CapEx or transport).

Validation checklist

- [ ] Mass balance: inflow × coefficient = outflow at every depot and refinery (error < 1e-6)
- [ ] Capacity: every facility at ⩽ 100% utilisation, and list any below 50%
- [ ] Demand: SAF received at each airport ⩾ D_a

- [ ] Supply: each site ships ⩽ its production; total ⩽ 6,800 kt
- [ ] Cost recomputation: recompute the objective independently from the solution; it matches the solver's value
- [ ] Optimality: MIP gap ⩽ 0.5%, solve time recorded

Tools: HiGHS (highspy or via Pyomo appsi), pytest (automated checks), JSON export.

9. Stage 6 Results analysis and sensitivity

First break down the cost structure, then sweep 8 key assumptions and a "number of refineries vs total cost" curve. The aim is to show under which assumptions the optimal plan is stable, and under which it flips.

Owner: C (calls B's build_model; no separate model)

Tasks

1. **Cost breakdown**: share of depot CapEx, refinery CapEx, OpEx and each transport leg; average cost per tonne of SAF (USD/t SAF).
2. **Operating metrics**: facility utilisation, flow-weighted average haul distance per leg.
3. **Fleet sizing** (formula below).
4. **One-at-a-time sensitivity**: move one parameter at a time; produce a tornado chart.
5. **Trade-off curve**: force the total number of refineries N = N_min to N_min + 4, solve each, and plot how CapEx and transport cost change with N.
6. **Robustness**: count how often each facility is selected across all scenarios; those at ⩾ 80% are "core facilities".

Output (what this delivers): scenarios.csv, cost breakdown table, tornado chart, trade-off curve, core facility list.

Sensitivity sweep list

Parameter	Base case	Sweep range	What it affects
Capacity basis	To confirm with organisers	Input / output	At least 7 refineries vs about 2; the largest effect
Annualisation	r = 8%, n = 20	r 6–10%, n 15–30	Heavier CapEx weight concentrates refineries

Parameter	Base case	Sweep range	What it affects
Conversion yield y	C researches literature	Literature low–high	Total SAF; refinery count under output basis
Depot mass ratio α	1.0	0.5–1.0	Below 0.833, depots move next to farms (derived in section 6)
Circuity factor τ	OSRM-calibrated value	1.2–1.6	Long-leg cost; whether plants move closer to demand
Airport demand split	By passenger traffic	Equal / by traffic / all at BPN	Refinery → airport flow pattern
TRK sea-crossing cost	Parameter	0 to high	Which eastern refinery supplies TRK
Must use all 6,800 kt	No	Yes / no	Whether remote small clusters are dropped (Tanah Laut 275 kt, Bulungan 189 kt)

Truck type and fleet sizing

The leg rates do not depend on truck type, so truck choice does not change total cost, only the number of trips. Derive it from flows after solving (q = payload, v = average speed, t_load = loading/unloading time, H = annual operating hours):

```
N^{\text{trips}}_{ij}=\left\lceil\frac{x_{ij}}{q}\right\rceil,\qquad \text{Fleet size}=\frac{\sum_{ij}N^{\text{trips}}_{ij}\,\big(2d_{ij}/v+t_{\text{load}}\big)}{H}
```

Scale: moving 6,800 kt from farms to depots takes about 756,000 trips/year with 9 t trucks, or about 2.27 million trips/year with 3 t trucks. Large-truck payloads are 3×, 1.67× and 2× the small ones on the three legs. Recommendation: use large trucks wherever roads allow; small trucks only on narrow last-mile farm roads.

Tools: pandas (scenario table), joblib (parallel scenarios), matplotlib/plotly (tornado chart, trade-off curve).

10. Stage 7 Visualisation and delivery

The final package is 5 core charts and 3 tables, all generated automatically from solution.json and scenarios.csv. When the model changes, re-running regenerates every chart, so charts and numbers never disagree.

Owner: D (A supplies the map base layer)

Deliverables

Deliverable	Question it answers	Data source	Tool
Interactive map: site bubbles sized by production, depots marked S/L, refineries, airports, flow lines with width by tonnage	Where facilities sit and how material moves	solution.json	folium
Sankey diagram: district → depot → refinery → airport	The material-flow network; tonnes conserved at each echelon	solution.json	plotly
Cost bridge (waterfall): baseline → MILP	Where the money goes, and where it beats the intuitive plan	solution.json + baseline	plotly
Trade-off curve: number of refineries N vs total cost	Why we build N plants	scenarios.csv	matplotlib
Tornado chart	Which assumption matters most	scenarios.csv	matplotlib
Facility table: type, coordinates, size, utilisation	The brief's "optimal coordinates" and "sizing decisions"	solution.json	pandas
Flow table: origin, destination, tonnes, distance, truck type, trips per leg	The brief's "routing"	solution.json	pandas
Assumption register	Which assumptions the conclusions rest on	assumptions.md	—

Tool choice

- Map: folium (offline HTML, simplest) vs kepler.gl (good for many flow lines, heavier set-up) vs plotly maps (matches the other charts). Choose folium: fewer than 500 points and lines, no need for kepler.

- Interactive dashboard (bonus): Streamlit, with sliders for r, n, τ, y that re-solve live. The model is small enough for HiGHS to solve within a demo. The static report stays the main deliverable to avoid demo failures.

Presentation storyline

1. Problem and data: feedstock skewed west (70.5% vs 29.5%); airports and feedstock don't line up
2. Method: three-echelon network + MILP + coordinate refinement
3. Optimal plan: map + Sankey
4. Cost and savings: bridge chart against the baseline
5. Robustness: tornado chart + core facilities
6. Recommendations and next steps

11. Open assumptions, risks and milestones

The biggest risk is not the algorithm but the parameters the brief leaves out. Send the 8 questions below to the organisers on day one; where no answer comes, proceed with the defaults in section 4 and cover them in the section 9 sensitivity analysis.

Questions for the organisers

- [ ] Is a refinery's or depot's "Production Capacity" biomass input or product output?
- [ ] What is the SAF conversion yield y? Does pre-processing at the depot change mass (α)?
- [ ] What is the annual SAF demand at each of the 4 airports?
- [ ] Is there any OpEx data?
- [ ] Should CapEx be annualised? With what discount rate and life?
- [ ] Must all 6,800 kt be used?
- [ ] Do trucks have a per-trip fixed cost, road weight limits or type restrictions?
- [ ] TRK is on an island: how should the sea crossing be priced?

Risks and mitigations

Risk	Impact	Mitigation
Capacity basis misread	The whole plan is invalid	Run both bases; show them side by side in the report
Straight-line distance underestimates roads	Transport cost too low; plan biased towards centralised plants	Calibrate τ with OSRM + τ sensitivity
No airport demand data	Refinery → airport flows unreliable	Multiple scenarios + a public assumption register
Too few candidates leads to a sub-optimal plan	Cost too high	Alternating refinement + a robustness check with all 80 sites as candidates
Extended model solves too slowly	Miss the deadline	Distance filtering, warm start, switch to Gurobi if needed
Coordinates land in water or protected areas	Plan cannot be built	Manual map review; add an exclusion-zone layer as an extension

Milestones (as shares of total time)

1. **M1, about 20%**: Stages 1–2 done, params.yaml frozen, distance matrices ready. Gate: organisers have answered, or the team has agreed to use the defaults.
2. **M2, about 60%**: Stages 3–5 done; the first optimal solution passes every check and beats the baseline.
3. **M3, about 85%**: Stage 6 done; core facility list settled.
4. **M4, 100%**: Stage 7 done; map, charts, report and deck final.